

Robótica e Automação - Trabalho Nº1

Leonardo Soares da Costa Tanaka - DRE: 121067652

Engenharia de Controle e Automação/UFRJ

Rio de Janeiro, Brasil

Novembro de 2024

1 Introdução

Este trabalho tem como objetivo explorar a modelagem cinemática de um sistema robótico composto por um braço robótico Kuka KR90 de 6 graus de liberdade e uma mesa posicionadora KUKA KP2 de 2 graus de liberdade. O efetuator do braço é uma tocha de soldagem, e a cinemática do sistema é descrita por meio de dois arquivos URDF: um para o braço e outro para a mesa. Com base nesses arquivos e na transformação homogênea T fornecida, será possível determinar os parâmetros de Denavit-Hartenberg (DH) de ambos os subsistemas (KR90 e KP2) e do sistema integrado KR90-KP2, visando descrever a cinemática direta e desenvolver o modelo no Matlab utilizando o toolbox de robótica de Peter Corke. O trabalho também inclui a validação da cinemática direta e inversa do sistema através de uma trajetória de waypoints pré-definidos, assim como a determinação e validação do Jacobiano do sistema.

2 Desenvolvimento

Foi considerado o sistema da figura composto por um braço robótico Kuka KR90 do tipo 6R e uma mesa posicionadora KUKA KP2 do tipo 2R.

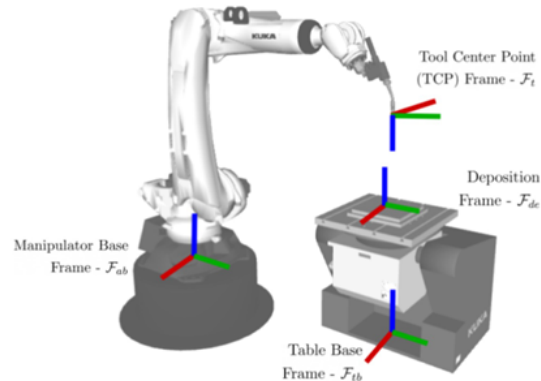


Figura 1: Braço robótico e mesa posicionadora

Foi considerado que o efetuador do braço manipulador é uma tocha para soldagem. Ademais, a cinemática deste sistema robótico é descrito em 2 arquivos urdf, um descrevendo a cinemática do KR90, kr90.urdf e outro descrevendo a cinemática da mesa posicionadora KP2, kp2.urdf. Além disso, foi considerado que a transformação homogênea do sistema de coordenadas da mesa F_{tb} com respeito a base do braço F_{ab} é dada por:

$$T_{at} = \begin{bmatrix} -1 & 0 & 0 & 1.73923 \\ 0 & -1 & 0 & 0.39198 \\ 0 & 0 & 1 & -0.46360 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

2.1 Parâmetros de Denavit-Hartenberg do braço robótico KR90

Para determinar os parâmetros de Denavit-Hartenberg do braço robótico KR90, foi necessário analisar o URDF desse braço robótico que pode ser descrito pela seguinte tabela:

Joint i	parent	child	xyz	rpy	type	axis
1	kr90_base_link	kr90_link_1	0, 0, 0.675	0, 0, 0	revolute	0, 0, -1
2	kr90_link_1	kr90_link_2	0.35, 0, 0	0, 0, 0	revolute	0, 1, 0
3	kr90_link_2	kr90_link_3	1.35, 0, 0	0, 0, 0	revolute	0, 1, 0
4	kr90_link_3	kr90_link_4	0, 0, -0.041	0, 0, 0	revolute	-1, 0, 0
5	kr90_link_4	kr90_link_5	1.2, 0, 0	0, 0, 0	revolute	0, 1, 0
6	kr90_link_5	kr90_link_6	0.215, 0, 0	0, 1.5708, 0	revolute	0, 0, -1
7	kr90_link_6	kr90_tool0	0.03046, 0.033, 0.43161	-0.2356, 0.8988, -0.0942	fixed	-

Tabela 1: URDF do braço robótico KR90

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	-0.675	0.35	$\pi/2$	revolute	0
2	θ_2	0	1.35	0	revolute	0
3	θ_3	0	0.041	$-\pi/2$	revolute	$\pi/2$
4	θ_4	-1.2	0	$\pi/2$	revolute	0
5	θ_5	0	0	$-\pi/2$	revolute	0
6	θ_6	-0.215	0	$-\pi$	revolute	0

Tabela 2: DH do braço robótico

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 6 para o efetuador a seguir:

$$T_{6e} = \begin{bmatrix} 0.61978941 & -0.09040281 & 0.77954372 & 0.03046 \\ -0.05855747 & 0.98524594 & 0.16081497 & 0.033 \\ -0.78258041 & -0.14531952 & 0.60535125 & 0.43161 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido os seguintes diagramas:

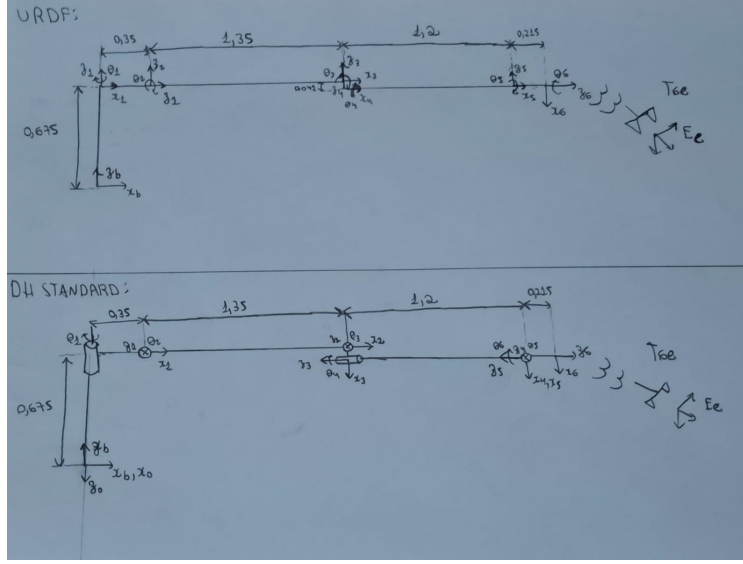


Figura 2: Diagramas do URDF e DH Standard do braço robótico

2.2 Parâmetros de Denavit-Hartenberg da mesa posicionadora KP2

Para determinar os parâmetros de Denavit-Hartenberg da mesa posicionadora KP2, foi necessário analisar o URDF dessa mesa posicionadora que pode ser descrita pela seguinte tabela:

Joint i	parent	child	xyz	rpy	type	axis
1	kp2_base_link	kp2_link_1	0, 0, 0.865	0, 3.1416, 0	revolute	1, 0, 0
2	kp2_link_1	kp2_link_2	0, 0, -0.22	0, 3.1416, 0	revolute	0, 0, 1
3	kp2_link_2	kp2_tool0	0, 0, 0	0, 0, 0	fixed	-

Tabela 3: URDF da mesa posicionadora KP2

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0.865 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	0	0	$-\pi/2$	revolute	$-\pi/2$
2	θ_2	0.22	0	0	revolute	$\pi/2$

Tabela 4: DH do braço robótico

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 2 para o efetuador a seguir:

$$T_{2e} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido os seguintes diagramas:

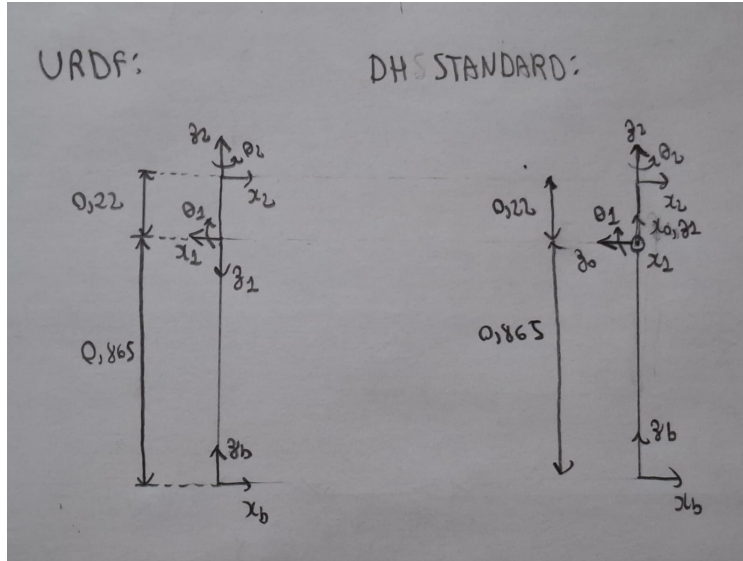


Figura 3: Diagramas do URDF e DH Standard da mesa posicionadora

2.3 Parâmetros de Denavit-Hartenberg dos sistemas KR90-KP2

Para determinar os parâmetros de Denavit-Hartenberg do sistema KR90-KP2, foi necessário analisar o URDF do braço robótico KP90, da mesa posicionadora KP2 e a transformação homogênea do sistema de coordenadas da mesa F_{tb} com respeito a base do braço F_{ab} , que estão respectivamente na Tabela 1, na Tabela 2 e na fórmula (1).

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	0.22	0	$\pi/2$	revolute	$\pi/2$
2	θ_2	1.73923	-0.39198	$-\pi/2$	revolute	0
3	θ_3	-0.27360	0.35	$\pi/2$	revolute	$\pi/2$
4	θ_4	0	1.35	0	revolute	0
5	θ_5	0	0.041	$-\pi/2$	revolute	$\pi/2$
6	θ_6	-1.2	0	$\pi/2$	revolute	0
7	θ_7	0	0	$-\pi/2$	revolute	0
8	θ_8	-0.215	0	$-\pi$	revolute	0

Tabela 5: DH do sistema KP90-KP2

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 8 para o efetuador a seguir:

$$T_{8e} = \begin{bmatrix} 0.61978941 & -0.09040281 & 0.77954372 & 0.03046 \\ -0.05855747 & 0.98524594 & 0.16081497 & 0.033 \\ -0.78258041 & -0.14531952 & 0.60535125 & 0.43161 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido o seguinte diagrama:

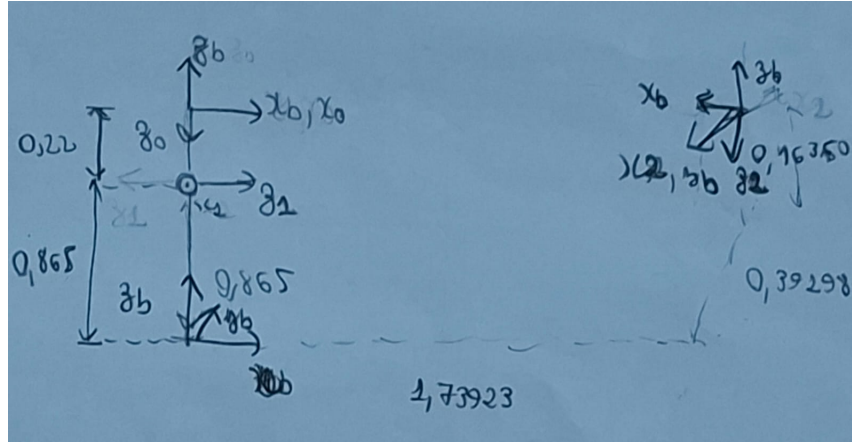


Figura 4: Diagrama do DH Standard do sistema KR90-KP2

2.4 Modelos dos sistemas

Utilizando o Robot Toolbox do Matlab foi construído os modelos utilizando a classe SerialLink de cada um dos 3 itens anteriores. Portanto, para realizar essa construção dos modelos, foi necessário utilizar a transformação homogênea da base para o eixo de coordenadas 0 (T_{b0}), o Denavit-Hartenberg do eixo de coordenadas 0 até o eixo de coordenadas do número de juntas (T_{0n}) e a transformação homogênea do último eixo de coordenadas do Denavit-Hartenberg até o eixo de coordenadas do efetuador (T_{ne}).

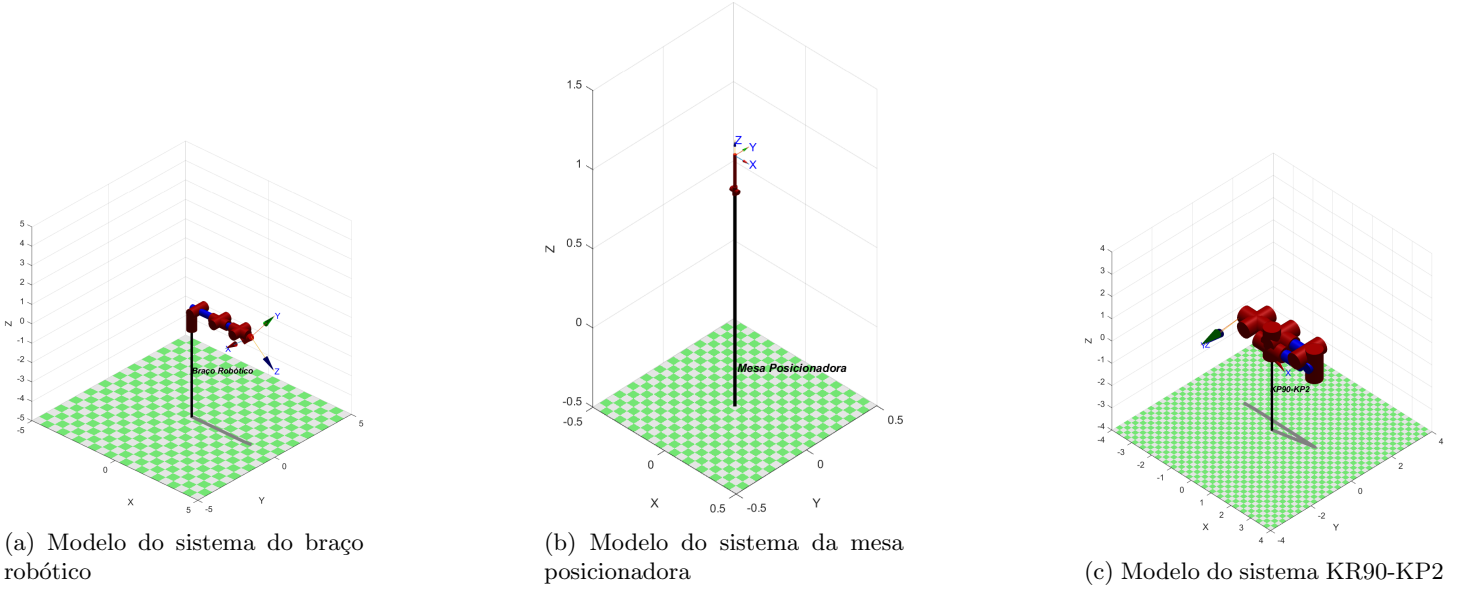


Figura 5: Modelos dos sistemas analisados. Cada subfigura apresenta um modelo específico utilizado no estudo.

2.5 Validação dos parâmetros de Denavit-Hartenberg

Foi feita a visualização de 4 configurações diferentes para cada modelo utilizando o Robot Toolbox do Matlab.

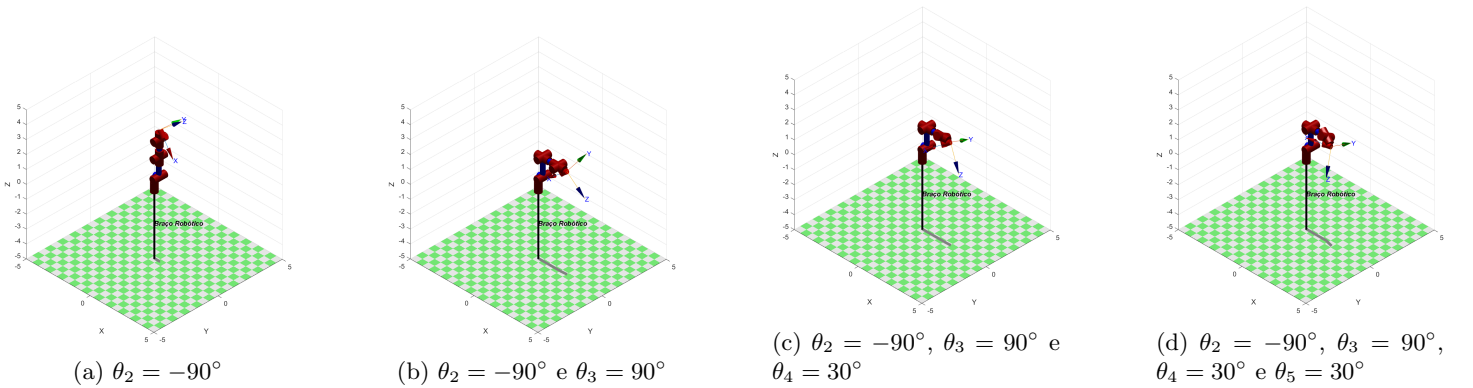


Figura 6: 4 configurações diferentes para o modelo do braço robótico.

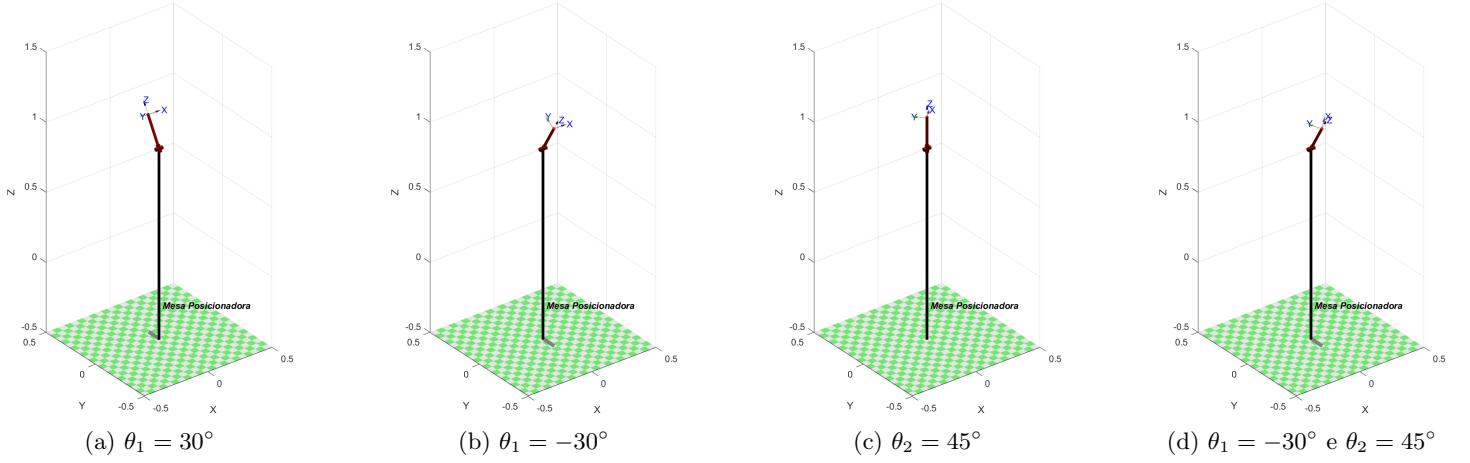


Figura 7: 4 configurações diferentes para o modelo da mesa posicionadora.

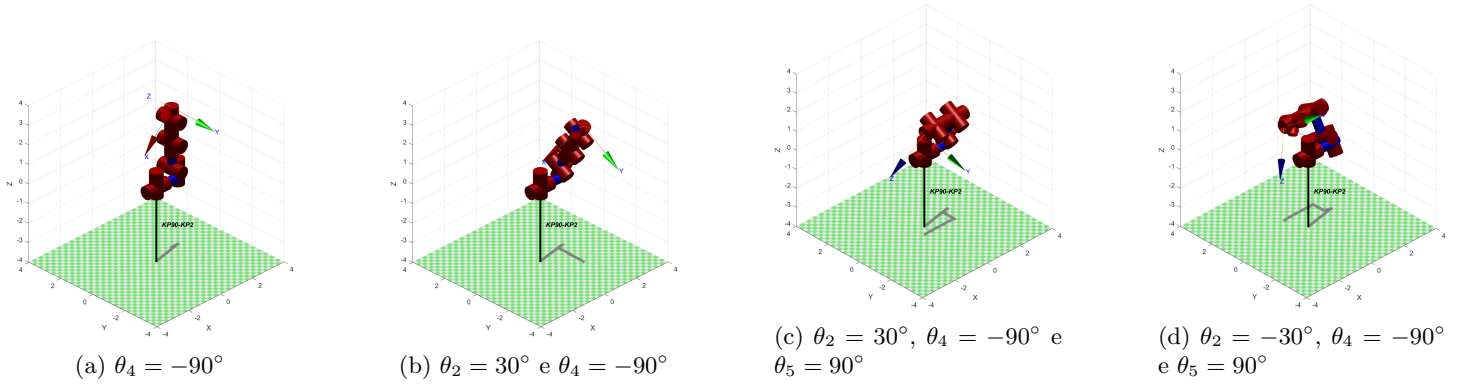


Figura 8: 4 configurações diferentes para o modelo do sistema KR90-KP2.

2.6 Verificação dos resultados da cinemática inversa

Neste estudo, foi explorado a função `ikine()` da *Robotics Toolbox* para calcular a cinemática inversa do robô serial Kr90, considerando uma trajetória em zigue-zague para o efetuador F_t em relação ao referencial F_{de} . Os *waypoints* utilizados são apresentados na Tabela 6. A orientação foi mantida constante para todos os casos, definida pelos ângulos de $roll = \pi$, $pitch = 0$ e $yaw = \pi/2$. Além disso, os ângulos das juntas da mesa posicionadora foram fixados em $\theta_t = [\pi/4, 0]$.

Waypoint	Posição p_{dt} (m)
1	$[0.04, 0, 0]^T$
2	$[-0.04, 0, 0]^T$
3	$[-0.04, 0.01, 0]^T$
4	$[0.04, 0.01, 0]^T$
5	$[0.04, 0.02, 0]^T$
6	$[-0.04, 0.02, 0]^T$
7	$[-0.04, 0.03, 0]^T$
8	$[0.04, 0.03, 0]^T$
9	$[0.04, 0.04, 0]^T$
10	$[-0.04, 0.04, 0]^T$

Tabela 6: *Waypoints* da trajetória em zigue-zague.

Os parâmetros da função `ikine()` foram explorados para cada *waypoint*, ajustando-se tolerância (`tol`), ângulos iniciais (`q0`), fator de amortecimento (`lambda`) e máscara de restrições (`mask`). Os resultados obtidos são apresentados a seguir:

Waypoint	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6
1	0.2438	-0.7993	2.0019	1.6157	-1.7229	-0.2979
2	0.2329	-0.7770	1.9295	1.5979	-1.7205	-0.3488
3	0.2368	-0.7713	1.9271	1.6020	-1.7219	-0.3453
4	0.2479	-0.7933	1.9995	1.6200	-1.7241	-0.2942
5	0.2519	-0.7873	1.9971	1.6244	-1.7254	-0.2906
6	0.2406	-0.7655	1.9247	1.6061	-1.7234	-0.3419
7	0.2445	-0.7598	1.9222	1.6102	-1.7248	-0.3385
8	0.2560	-0.7813	1.9945	1.6287	-1.7266	-0.2870
9	0.2600	-0.7753	1.9919	1.6331	-1.7278	-0.2835
10	0.2484	-0.7540	1.9197	1.6142	-1.7262	-0.3352

Tabela 7: Valores dos ângulos das juntas (θ_1 a θ_6) para cada *waypoint*.

2.7 Determinação do Jacobiano de F_t com respeito a F_{de}

Para determinar o Jacobiano de F_t com respeito a F_{de} , em termos do Jacobiano do braço robótico KR90 (J_a) e do Jacobiano da mesa posicionadora KP2 (J_t), foi levado em consideração que o jacobiano no ponto E (J_E), em função de J_n é dado por:

$$J_E = \begin{bmatrix} \mathcal{I} & -\vec{p}_{nE} \times \\ 0 & \mathcal{I} \end{bmatrix} J_n \quad (8)$$

e outras propriedades de troca do sentido do jacobiano e da independência da parte do jacobiano das juntas posteriores em relação às juntas anteriores. Além disso, teve que ser usado uma matriz permutação (P) já que o θ_1 e θ_2 da mesa posicionadora são, respectivamente, o θ_2 e θ_1 do sistema KR90-KP2. Assim obtendo o Jacobiano de F_t com respeito a F_{de} :

$$J_{dt} = \begin{bmatrix} - \begin{bmatrix} \mathcal{I} & -\vec{p}_{dt} \times \\ 0 & \mathcal{I} \end{bmatrix} J_t P & J_a \end{bmatrix} \quad (9)$$

2.8 Validação do Jacobiano de F_t com respeito a F_{de}

Para validar o Jacobiano de F_t com respeito a F_{de} foi feito um código em Matlab que obtém o Jacobiano do sistema KR90-KP2 em relação ao eixo de coordenadas do efetuador pelo método `jacobe` e em relação ao eixo de coordenadas do topo da mesa pelo método `jacob0` do `SerialLink` do DH desse sistema. Além disso, foi feito nesse mesmo código esse jacobiano usando a implementação da equação obtida acima com o Jacobiano do braço robótico KR90 em relação ao eixo de coordenadas do efetuador, o Jacobiano da mesa posicionadora KP2 em relação ao eixo de coordenadas do topo da mesa e o jacobiano geométrico associado a transformação homogênea do topo da mesa até o efetuador com a função `tr2jac`. Isso foi feito dessa forma, porque a equação obtida acima está no abstrato e teve que ser passado para ser relacionado ao eixo de coordenadas do efetuador ou ao eixo de coordenadas do topo da mesa. Assim, tendo o resultado do Jacobiano do sistema KR90-KP2 em relação ao eixo de coordenadas do efetuador e em relação ao eixo de coordenadas da base em ambos os métodos sendo:

$$(J_{dt})_{F_t} = \begin{bmatrix} -0.0400 & -0.2200 & 1.2015 & 0.8580 & 0.0272 & 0.1084 & 0.4094 & 0.0222 \\ -0.0000 & -0.0000 & -0.2364 & 0.1146 & 1.0537 & 0.5958 & -0.2400 & -0.0330 \\ 0.0000 & 0.0000 & -1.2016 & 0.8983 & 0.3979 & 0.2171 & 0.4396 & 0.0208 \\ 0.0000 & -0.0001 & 0.7071 & -0.6862 & -0.6862 & -0.7212 & 0.1259 & 0.7826 \\ 0.0000 & 1.0000 & 0.0001 & -0.2414 & -0.2414 & 0.3492 & 0.9153 & 0.1453 \\ 1.0000 & -0.0000 & 0.7071 & 0.6862 & 0.6862 & -0.5983 & 0.3825 & -0.6053 \end{bmatrix} \quad (10)$$

$$(J_{dt})_{F_{de}} = \begin{bmatrix} 0.0000 & 0.0000 & -0.2364 & 0.1145 & 1.0537 & 0.5958 & -0.2401 & -0.0330 \\ -0.0400 & -0.2200 & 1.2015 & 0.8580 & 0.0273 & 0.1085 & 0.4094 & 0.0222 \\ 0.0000 & -0.0000 & 1.2016 & -0.8983 & -0.3980 & -0.2171 & -0.4396 & -0.0208 \\ 0.0000 & 1.0000 & -0.0000 & -0.2414 & -0.2414 & 0.3493 & 0.9153 & 0.1453 \\ 0.0000 & 0.0000 & 0.7071 & -0.6862 & -0.6862 & -0.7212 & 0.1260 & 0.7826 \\ -1.0000 & -0.0000 & -0.7071 & -0.6862 & -0.6862 & 0.5983 & -0.3825 & 0.6053 \end{bmatrix} \quad (11)$$

2.9 Implementação da solução da cinemática inversa pelo método iterativo

Para resolver a cinemática inversa do robô KR90-KP2, utilizou-se um método iterativo baseado na Jacobiana do manipulador. A seguir, são descritos os principais passos do algoritmo desenvolvido e as ferramentas empregadas. Os parâmetros iniciais do algoritmo incluem a configuração inicial das juntas ($\mathbf{q}_0$), a posição desejada do efetuador ($\mathbf{x}_d$), um ganho para o ajuste do vetor de deslocamento (γ), a precisão do critério de parada (ϵ) e o número máximo de iterações permitido ($n_{\max}$). Esses parâmetros garantem que o método seja robusto e ajustável às condições do problema. O método iterativo baseia-se na seguinte formulação:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \gamma \cdot \mathbf{J}_k^\dagger (\mathbf{x}_d - \mathbf{x}_k), \quad (12)$$

onde:

- $\mathbf{q}_k$ é o vetor de ângulos das juntas na iteração k ;
- γ é o ganho escalar;
- $\mathbf{J}_k^\dagger$ é a pseudo-inversa da Jacobiana $\mathbf{J}_k$;
- $\mathbf{x}_d$ é a posição desejada do efetuador;
- $\mathbf{x}_k$ é a posição do efetuador calculada a partir da configuração $\mathbf{q}_k$.

A cada iteração, a Jacobiana no espaço cartesiano é obtida com a função `jacob0`, sendo considerada apenas sua submatriz de posição ($\mathbf{J}_{\text{pos}}$). A pose atual do efetuador é calculada com a função `fkine`, que retorna a matriz de transformação homogênea correspondente à configuração atual das juntas.

O critério de parada é baseado na norma do vetor de erro entre a posição desejada e a posição atual, dada por:

$$\|\mathbf{x}_d - \mathbf{x}_k\| < \epsilon, \quad (13)$$

onde ϵ é o valor de precisão especificado. Caso o erro seja inferior ao limite estabelecido, considera-se que o algoritmo convergiu.

A seguir, foi preenchida a tabela com os resultados obtidos após a execução do algoritmo para o waypoint 1 do item 6 ($p_{dt} = [0.04, 0, 0]^T m$):

- **Ângulos finais das juntas ($\mathbf{q}_k$):** $\begin{bmatrix} -0.2503 & 0.0244 & 1.6124 & 0.9254 & -1.7375 & -0.0813 & -0.0147 & 0.0416 \end{bmatrix}^T$ rad.
- **Número de iterações:** 31.
- **Erro final alcançado:** 1×10^{-6} m.

A convergência do algoritmo foi analisada a partir da evolução do erro entre a posição desejada ($\mathbf{x}_d$) e a posição calculada ($\mathbf{x}_k$) ao longo das iterações.

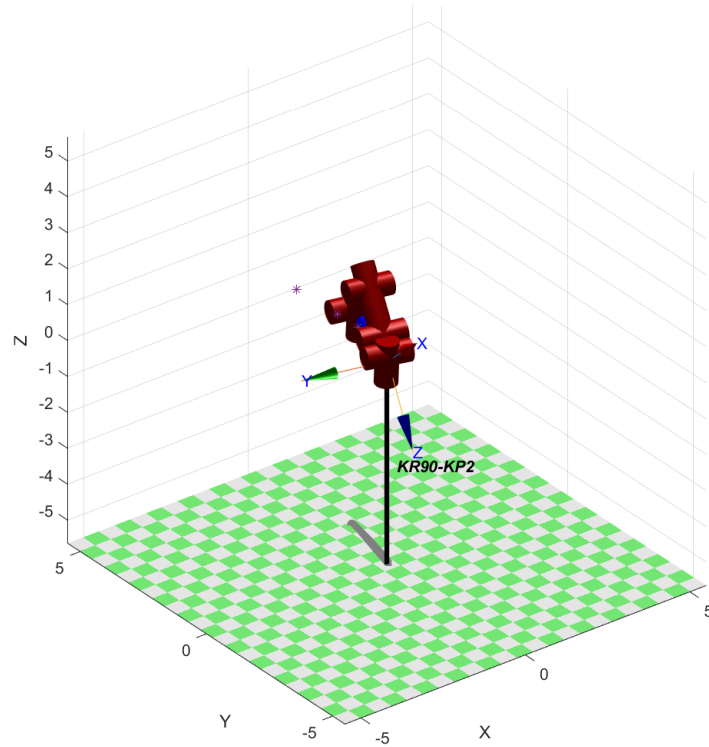


Figura 9: Solução da cinemática inversa pelo método iterativo

Esses resultados validam o algoritmo e demonstram sua eficácia na resolução da cinemática inversa do robô KR90-KP2, alcançando a posição desejada com alta precisão.

3 Conclusão

Este trabalho apresentou uma análise completa do sistema robótico composto pelo braço KUKA KR90 e a mesa posicionadora KUKA KP2, integrando conceitos teóricos e práticos de modelagem cinemática. Os principais resultados incluem:

- Determinação e validação dos parâmetros Denavit-Hartenberg para os subsistemas KR90, KP2 e o sistema combinado.
- Simulações com a *Robot Toolbox* no MATLAB, utilizando a classe `SerialLink`.
- Solução da cinemática inversa com a função `ikine()`.
- Cálculo e validação do Jacobiano composto.
- Implementação de um algoritmo iterativo de cinemática inversa.

Assim, o trabalho consolidou conhecimentos teóricos e computacionais aplicados à modelagem de sistemas robóticos multijuntas.

Apêndice

Listing 1: Código do braço robótico (KR90)

```

1 % Parâmetros DH do robô
2 L1 = Revolute('d', -0.675, 'a', 0.35, 'alpha', pi/2, 'offset', 0);      % Elo 1
3 L2 = Revolute('d', 0, 'a', 1.35, 'alpha', 0, 'offset', 0);            % Elo 2
4 L3 = Revolute('d', 0, 'a', 0.041, 'alpha', -pi/2, 'offset', pi/2);     % Elo 3
5 L4 = Revolute('d', -1.2, 'a', 0, 'alpha', pi/2, 'offset', 0);          % Elo 4
6 L5 = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', 0);           % Elo 5
7 L6 = Revolute('d', -0.215, 'a', 0, 'alpha', -pi, 'offset', 0);        % Elo 6
8
9 % Definir o robô
10 kukakr90 = SerialLink([L1 L2 L3 L4 L5 L6], 'name', 'Braço Robótico');
11
12 % Transformação homogênea da base para o eixo 0 (T_b0)
13 T_b0 = [1, 0, 0, 0;
14         0, -1, 0, 0;
15         0, 0, -1, 0;
16         0, 0, 0, 1];
17
18 T_6e = transl(0.03046, 0.033, 0.43161) * trotx(-5.4) * troty(51.5) * trotx(-13.5);
19
20 % Incorporar as transformações no robô
21 kukakr90.base = SE3(T_b0);      % Define a base do robô
22 kukakr90.tool = T_6e;           % Define a ferramenta (tool) do robô
23
24 % Configuração inicial das juntas (todas em zero)
25 q0 = zeros(1, 6);
26
27 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
28 T_be = kukakr90.fkine(q0);
29
30 % Exibir a transformação final do efetuador
31 disp('Transformação do efetuador em relação à base (T_be):');
32 disp(T_be.T);
33
34 % Plotar o robô na configuração inicial com controles interativos
35 figure;
36 kukakr90.teach(q0, 'workspace', [-5 5 -5 5 -5 5]);

```

Listing 2: Código da mesa poscionadora (KP2)

```

1 % Parâmetros DH do robô
2 L1 = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', -pi/2); % Elo 1
3 L2 = Revolute('d', 0.22, 'a', 0, 'alpha', 0, 'offset', pi/2); % Elo 2
4
5 % Definir o robô
6 kukakp2 = SerialLink([L1 L2], 'name', 'Mesa Posicionadora');
7
8 % Transformação homogênea da base para o eixo 0 (T_b0)
9 T_b0 = [0, 0, -1, 0;
10         0, 1, 0, 0;
11         1, 0, 0, 0.865;
12         0, 0, 0, 1];
13
14 % Transformação homogênea do eixo 2 para o efetuador (T_2e)
15 T_2e = eye(4); % Matriz identidade, conforme especificado
16
17 % Incorporar as transformações no robô
18 kukakp2.base = SE3(T_b0); % Define a base do robô
19 kukakp2.tool = SE3(T_2e); % Define a ferramenta (tool) do robô
20
21 % Configuração inicial das juntas (todas em zero)
22 q0 = zeros(1, 2);
23
24 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
25 T_be = kukakp2.fkine(q0);
26
27 % Exibir as transformações
28 disp('Transformação da base para o efetuador (T_be):');
29 disp(T_be.T);
30
31 % Plotar o robô na configuração inicial com controles interativos
32 figure;
33 kukakp2.teach(q0, 'workspace', [-0.5 0.5 -0.5 0.5 -0.5 1.5]);

```

Listing 3: Código do sistema (KR90-KP2)

```

1 % Parâmetros DH do sistema KP90-KP2
2 L(1) = Revolute('d', 0.22, 'a', 0, 'alpha', pi/2, 'offset', pi/2); % Elo 1
3 L(2) = Revolute('d', 1.73923, 'a', -0.39198, 'alpha', -pi/2, 'offset', 0); % Elo 2
4 L(3) = Revolute('d', -0.27360, 'a', 0.35, 'alpha', pi/2, 'offset', pi/2); % Elo 3
5 L(4) = Revolute('d', 0, 'a', 1.35, 'alpha', 0, 'offset', 0); % Elo 4
6 L(5) = Revolute('d', 0, 'a', 0.041, 'alpha', -pi/2, 'offset', pi/2); % Elo 5
7 L(6) = Revolute('d', -1.2, 'a', 0, 'alpha', pi/2, 'offset', 0); % Elo 6
8 L(7) = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', 0); % Elo 5
9 L(8) = Revolute('d', -0.215, 'a', 0, 'alpha', -pi, 'offset', 0); % Elo 6
10
11 % Definir o robô
12 kukakr90kp2 = SerialLink(L, 'name', 'KP90-KP2');
13
14 T_8e = transl(0.03046, 0.033, 0.43161) * trotx(-5.4) * troty(51.5) * trotx(-13.5);
15
16 % Incorporar as transformações no robô
17 kukakr90kp2.base = trotx(180); % Define a base do robô
18 kukakr90kp2.tool = T_8e; % Define a ferramenta (tool) do robô
19
20 % Configurar a configuração inicial das juntas (todas em zero)
21 q0 = zeros(1, 8);
22
23 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
24 T_be = kukakr90kp2.fkine(q0);
25
26 % Exibir as transformações
27 disp('Transformação da base para o efetuador (T_be):');
28 disp(T_be.T);
29
30 % Plotar o robô na configuração inicial com controles interativos
31 figure;
32 kukakr90kp2.teach(q0, 'workspace', [-4 4 -4 4 -4 4]);

```

Listing 4: Código da Cinemática Inversa

```

1 % Configuração dos Waypoints
2 waypoints = [ 0.04, 0.00, 0.0; -0.04, 0.00, 0.0; -0.04, 0.01, 0.0; 0.04, 0.01, 0.0; 0.04, 0.02, 0.0;
3             -0.04, 0.02, 0.0; -0.04, 0.03, 0.0; 0.04, 0.03, 0.0; 0.04, 0.04, 0.0; -0.04, 0.04, 0.0;];
4 % ----- Configuração do Robô Kuka KR90 -----
5 % ----- Configuração do Robô Kuka KP2 (Mesa Posicionadora) -----
6 % ----- Configuração do Sistema KR90-KP2 -----
7 % ----- Configuração da Transformação entre Robôs -----
8 % Transformação entre as bases dos robôs
9 T_Fab_Ftb = SE3;
10 T_Fab_Ftb.t = [1.73923, 0.39198, -0.46360]; % Translação
11 T_Fab_Ftb = T_Fab_Ftb * SE3.Rz(180); % Rotação de 180 graus em Z
12 % Angulos conhecidos da mesa posicionadora
13 theta_t = [pi/4, 0];
14 T_Ftb_Fde = kukaKp2.fkine(theta_t); % Cinemática direta do KP2
15 % ----- Cálculo da Cinemática Inversa e Trajetoria -----
16 % Configurações iniciais
17 q0 = zeros(1, kukaKr90.n); % Angulos iniciais das juntas
18 tol = 1e-6; % Tolerância para solução da cinemática inversa
19 lambda = 0.1; % Taxa de regularização
20
21 % Matriz para armazenar as soluções das juntas
22 q_solutions = zeros(size(waypoints, 1), kukaKr90.n);
23
24 % Iteração para cada waypoint
25 for i = 1:size(waypoints, 1)
26     % Define a transformação desejada para o waypoint atual
27     T_Fde_Ft = SE3.Rz(90) * SE3.Rx(180); % Orientação fixa
28     T_Fde_Ft.t = waypoints(i, :); % Posição do waypoint
29
30     % Transformação total em relação a base absoluta
31     T_Fab_Ft = T_Fab_Ftb * T_Ftb_Fde * T_Fde_Ft;
32
33     % Calcula a cinemática inversa
34     q_solutions(i, :) = kukaKr90.ikine(T_Fab_Ft, q0, 'tol', tol, 'lambda', lambda, 'mask', [1 1 1 1 1 1]);
35
36     % Valida a solução obtida com a cinemática direta
37     T_check = kukaKr90.fkine(q_solutions(i, :));
38     disp(['Waypoint ', num2str(i)]);
39     disp('Transformação homogênea obtida:');
40     disp(kukaKr90kp2.fkine([0, pi/4, q_solutions(i,:) ]));
41     disp('Angulos das juntas:');
42     disp(q_solutions(i, :));
43 end

```

Listing 5: Código do Jacobiano de F_t com respeito a F_{de}

```

1 % ----- Configuração do Robô Kuka KR90 -----
2 % ----- Configuração do Robô Kuka KP2 (Mesa Posicionadora) -----
3 % ----- Configuração do Sistema KR90-KP2 -----
4 % ----- Configuração da Transformação entre Robôs -----
5 % Angulos conhecidos da mesa posicionadora (KP2)
6 theta_t = [pi/4, 0];
7 T_Ftb_Fde = kukaKp2.fkine(theta_t); % Calcula a cinemática direta da mesa posicionadora (KP2)
8 % Angulos fornecidos para o robô KUKA KR90 (em radianos)
9 theta = [0, pi/4, 0.2438, -0.7993, 2.0019, 1.6157, -1.7229, -0.2979];
10 % Calcula o Jacobiano do robô KUKA KR90 em relação à ferramenta
11 J_tool = kukakr90kp2.jacobe(theta); disp(J_tool);
12 % Calcula o Jacobiano do robô KUKA KR90 em relação à base
13 J_base = kukakr90kp2.jacob0(theta); disp(J_base);
14
15 % Matriz de permutação P para ajustar a ordem de colunas
16 P = [0 1; 1 0];
17
18 % Vetor p_ne (vetor posição da ferramenta no espaço cartesiano)
19 p_dt = [0.04; 0; 0];
20 % Matriz anti-simétrica associada ao vetor p_dt
21 p_dt_cross = [
22     0      -p_dt(3)  p_dt(2);
23     p_dt(3)  0      -p_dt(1);
24     -p_dt(2)  p_dt(1)  0];
25 % Construção da matriz adjunta (adj_matrix)
26 adj_matrix = [
27     eye(3),      -p_dt_cross; % Parte superior: identidade e anti-simétrica
28     zeros(3), eye(3)        ]; % Parte inferior: identidade
29
30 % Jacobiano do robô KUKA KR90 em relação à ferramenta (considerando apenas seus ângulos)
31 J_kukakr90_tool = kukakr90.jacobe(theta(3:8));
32
33 % Jacobiano da mesa posicionadora (KP2) em relação à base
34 J_kukaKp2_base = kukaKp2.jacobe(theta_t);
35
36 % Jacobiano transformado para o sistema T_Fde_Ft
37 J_T_Fde_Ft = tr2jac(inv(T_Ftb_Fde.T) * inv(T_Fab_Ftb.T) * kukakr90.fkine(theta(3:8)).T);
38
39 % Concatenação final do Jacobiano em relação à ferramenta
40 J_tool = [J_T_Fde_Ft * adj_matrix * -J_kukaKp2_base * P, J_kukakr90_tool]; disp(J_tool);
41 % Concatenação final do jacobiano em relação à ferramenta
42 J_base = [adj_matrix * -J_kukaKp2_base * P, J_T_Fde_Ft * J_kukakr90_tool]; disp(J_base);

```


Listing 6: Código do método iterativo da solução da cinemática inversa

```

1 % Parâmetros iniciais
2 xd = [0.04; 0; 0]; % Posição desejada do efetuador
3 q0 = [0; 0; 0; pi/2; -pi/2; 0; 0; 0]; % Ângulos iniciais das juntas em radianos
4 qk = q0; % Inicializando qk
5 Q = [qk]; % Armazenar a evolução dos ângulos
6 ganho = 0.4; % Ganho do algoritmo
7 precisao = 1e-6; % Critério de parada
8 max_iter = 100; % Máximo de iterações
9 % ----- Configuração do Sistema KR90-KP2 -----
10 % Loop para resolver a cinemática inversa iterativamente
11 for i = 1:max_iter
12     % Calcula a pose atual (transformação homogênea)
13     Tk = kukakr90kp2.fkine(qk); % Transformação homogênea atual
14     xk = Tk.t; % Extraíndo a posição atual (x, y, z)
15     % Verificando o critério de parada
16     erro = norm(xd - xk);
17     fprintf('Iteração %d: Erro = %.6f\n', i, erro);
18     if erro < precisao
19         disp('Convergência alcançada!');
20         break;
21     end
22     % Jacobiana no espaço cartesiano
23     Jk = kukakr90kp2.jacob0(qk); % Jacobiana 6x8 no espaço cartesiano
24     Jk = Jk(1:3, :); % Extraíndo as linhas de posição (3x8)
25     % Atualização dos ângulos das juntas
26     dqk = ganho * pinv(Jk) * (xd - xk); % Usando pseudo-inversa para evitar singularidades
27     qk = qk + dqk; % Atualização dos ângulos
28     % Armazenar a solução
29     Q = [Q qk];
30 end
31
32 % Exibir os valores finais dos ângulos
33 disp('Valores finais dos ângulos (rad):');
34 disp(qk);
35 % Calcular e exibir a posição final do efetuador
36 Tk_final = kukakr90kp2.fkine(qk); % Calcular a transformação final
37 xk_final = Tk_final.t; % Posição final (x, y, z)
38 disp('Posição final do efetuador (m):'); disp(xk_final);
39 % Plot da evolução
40 kukakr90kp2.plot(Q, 'delay', 0.5, 'trail', '*');

```